

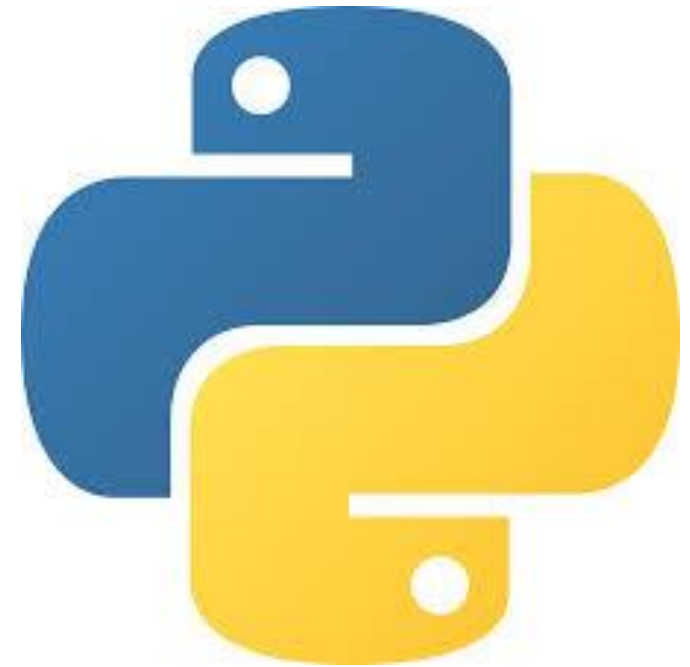
Automating Excel Spreadsheets with Python

BY JON PYLES

OECD 2026

What is Python?

- Python is a free and open-source programming language.
- It is one of the most widely used language
 - [TIOBE Index](#)
 - [Stack Overflow](#)
- It is easy to read and write compared to other programming languages.
- Extensive Library of pre-built modules you can import into any project
 - Data Analysis
 - Encryption
 - Web Applications
 - And many more!



Python Libraries for Data Analysis and Automation

- There are many Python libraries available for created/edited spreadsheets:
 - Pandas – a fast and powerful data analysis and manipulation tool
 - Openpyxl – designed for reading/writing to Excel files and has access to almost all tools available within Excel.
 - Csv – designed for reading and writing csv files
 - Gspread – design for reading/writing google sheets
 - Email and SMTPlib – libraries for sending and receiving emails
 - Streamlit – displaying interactive data onto web applications
- Helpful links:
 - Python for Beginners: <https://www.python.org/about/gettingstarted/>
 - Python Tutorials: <https://www.w3schools.com/python/>
 - Automate the boring stuff: <https://automatetheboringstuff.com/>

Project 1 – Automating Account Tracking and Creating Password Alerts

- Problem 1: Users were letting their USAS/USPS passwords expire.
 - Problem 2: Auditors wanted a list of all User accounts, roles, and MFA status for all USAS/USPS applications.
 - Solution: Generated daily USAS/USPS user-listing reports across all applications and sent them via FTP to a server. A Python script then processed each file to identify upcoming password expirations and wrote the user information to a master user listing file.
- The Pandas library is used to isolate each user and their password expiration date.
 - The Email and SMTP library sends emails to users whose passwords expire in 7 days.
 - Openpyxl is used to create one master file of all users.

```
users_df = pd.read_csv(input_file_path) #creates a dataframe from the input csv file containing user data
usernames_list = users_df['Username'].tolist() #creates a list of usernames from the dataframe.
```

Figure 1: Using Pandas to isolate the usernames out of a csv file

Project 2 – Automating the Creation of Excel Graphs and Data Clean Up

- Problem 1: Districts wanted visual representations of their financial data to present to their board.
- Problem 2: Some generated reports required significant cleanup before they could be used effectively.

A	B	C	D	E	F	G	H
Full Account	Description	Active	MTD Received	MTD Expended	Month End Balance	Month Name	Fiscal Year
001-0000	GENERAL FUND	TRUE	[2054442.02, 760288.38, 800024.10, 83958669.77, 1048180.86, 944780.01, 888220181.05, 7932288.57, 7787532.66,			[July, August, September, October, November, December]	2022
001-0000	GENERAL FUND	TRUE	[1995530.48, 776006.48, 854310.89, 11974336.55, 1051875.93, 929589.32, 918610641.39, 8334771.94, 8259493.51,			[July, August, September, October, November, December]	2023
001-0000	GENERAL FUND	TRUE	[1781685.65, 1313486.58, 855540.05, 11081674.54, 1535292.22, 1242577.51,			[July, August, September, October, November, December]	2024
001-0000	GENERAL FUND	TRUE	[2413728.95, 1259751.38, 1040544.96, 1180279.82, 1156989.73, 1067001.03,			[July, August, September, October, November, December]	2025

Figure 2: Sample Cash Account History Report from USAS

- Solution: Developed a Python program that cleans and formats the data into an easy-to-read Excel file, automatically generates charts, and emails the completed report to the district Treasurer.
 - Uses pandas to isolate bracketed cells and extract their numeric values, then uses openpyxl to write the cleaned data to a new Excel file and generate charts within that file.

Project 2 Bar Chart Example

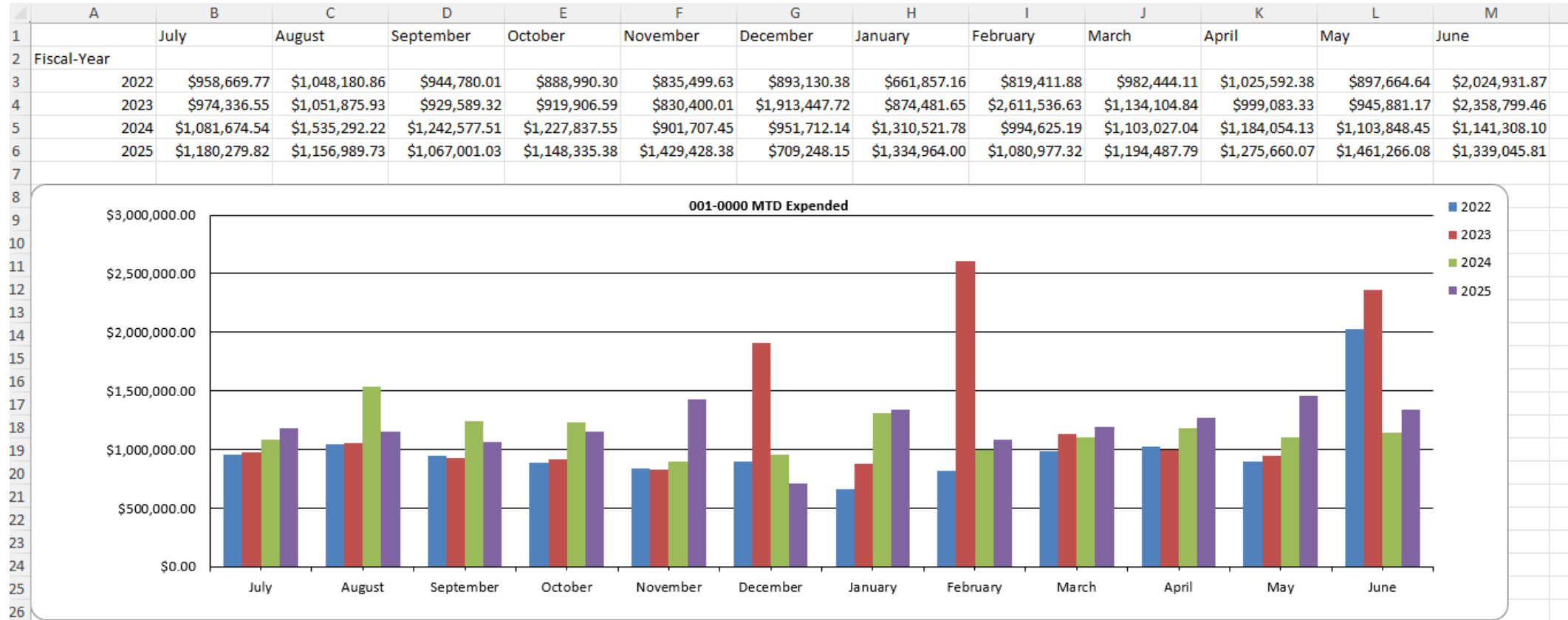


Figure 3: A bar chart of expenditures created after the script has finished running

Project 2 – Example Line Graph Code

```
#Example of how to create a line chart
def example_line_chart(output_file:str):
    print("Example Chart")
    # Sample data
    data = [
        ["Month", "2023"],
        ["Jan", 10],
        ["Feb", 15],
        ["Mar", 12],
        ["Apr", 18],
        ["May", 20],
    ]

    wb = Workbook()
    ws = wb.active

    for row in data:
        ws.append(row)

    data_values = Reference(ws, min_col=2, min_row=1, max_col=2, max_row=len(data))
    categories = Reference(ws, min_col=1, min_row=2, max_col=1, max_row=len(data))

    chart = LineChart()
    chart.title = "Example Line Chart"
    chart.style = 20
    chart.add_data(data_values)
    chart.set_categories(categories)

    ws.add_chart(chart, "D1") # Place chart in Cell D1
    wb.save(output_file)
    print("example.xlsx created. Open it to see the chart.")
```

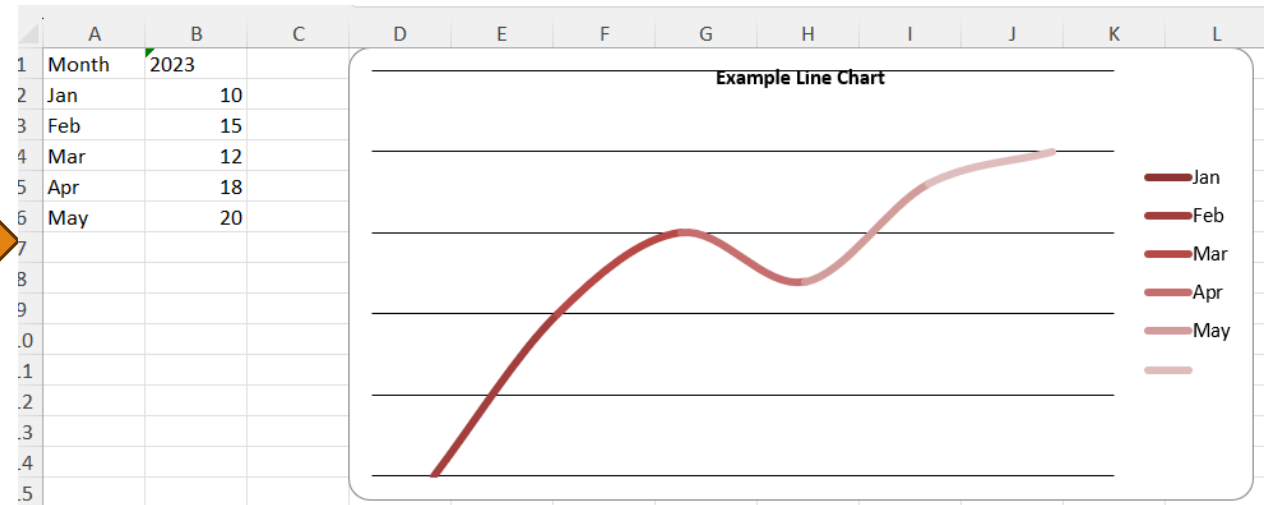
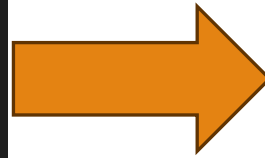


Figure 5: Output of example code

Figure 4: Example code on how to create a line chart using Python and Openpyxl charts library

Automated Workflow

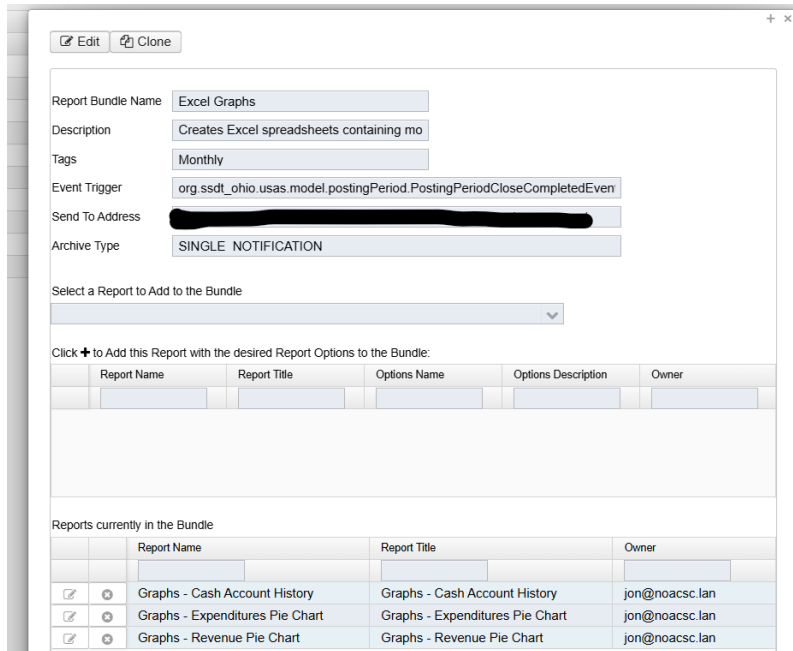


Figure 6: Screenshot of Report Bundle used for graphs

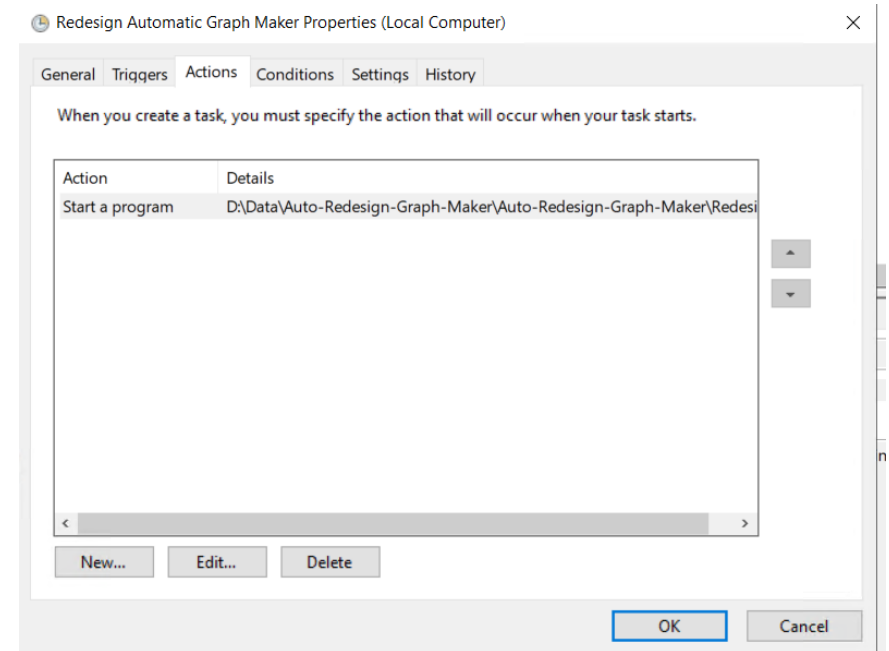


Figure 7: Screenshot of Windows Task Scheduler

- When a treasurer closes a month, a report bundle runs sending files to one of our servers via FTP.
- Windows Task Scheduler is used to set when the Python Scripts should run.

Using AI to Create Python Scripts

- AI tools can generate Python scripts from plain-language instructions, allowing anyone to automate tasks without deep programming knowledge.
- AI can suggest libraries, write functions, debug code, and optimize scripts based on best practices.
 - Some libraries documentation are hundreds of pages long. AI can break down just the functions you need.
- AI tools for Programming:
 - - Copilot
 - - Google Gemini
 - - ChatGPT

Using AI to Create Python Scripts (cont.)

- Provide clear and precise prompts of what you want the script to do.
- List the layout of your input and output files. If there are specific headers you want the script to look through list them in the prompt.
- Ask it to give step by step instructions and to include comments explaining the code
- DO NOT ENTER SENSITIVE INFORMATION IN THE PROMPT!
- Remember, AI is not always accurate. You will probably have to make slight changes to the AI provided code to fit your needs.

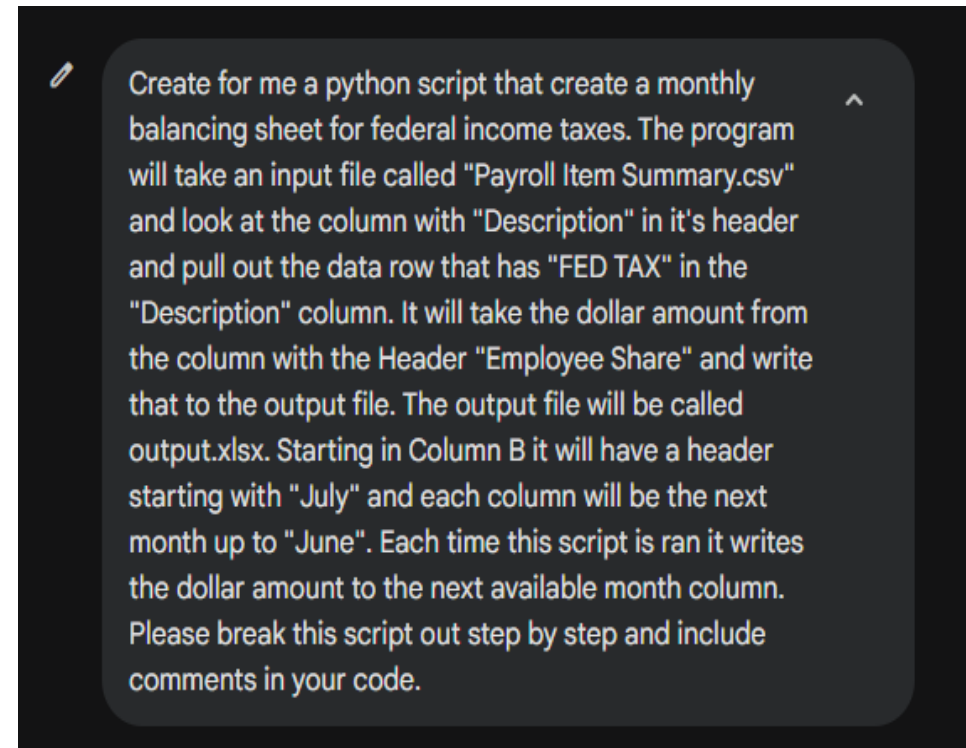


Figure 8: Example of entering a prompt in Gemini

Git Repo

Full projects are available on Github: <https://github.com/JPyles01/OECN-2026-Projects-Repository/tree/main/OECN-2026-Automating-Excel-Spreadsheets-Public>



If you have any questions, you can contact me at: jon@noacsc.org